

Spot the Fake Photo - Technical Report

1. How I Did It

APPROACH: Transfer Learning with MobileNetV3-Small

The task is to detect whether an image is a real photo or a photo of a screen (a 'recapture'). There is no object to recognise - the clue is in subtle visual artefacts: Moire patterns, pixel grids, screen glare, and slightly-off colours.

I fine-tuned a MobileNetV3-Small model (pre-trained on ImageNet). Its convolutional filters already detect high-frequency textures, making it ideal for spotting screen artefacts without needing thousands of training images.

Two-Phase Training Strategy:

Phase 1 (20 epochs, LR=1e-3): Froze the entire ImageNet backbone. Trained only the classification head using Adam + CosineAnnealing LR scheduler.

Phase 2 (25 epochs, LR=3e-5): Unfroze the last 4 MobileNet feature blocks to fine-tune texture detection specifically for screen/Moire artefacts.

Regularization to prevent overfitting on 100 images:

- Dropout 0.5 in the classifier head
- L2 weight decay = 1e-4
- Label smoothing = 0.1 (prevents overconfident memorization)
- Aggressive augmentation: random crop, flip, rotation, color jitter, grayscale
- Best-epoch checkpointing: saved weights at the epoch with highest accuracy

2. Accuracy

Overall Accuracy: 98.0% (98 / 100 images correct)

Breakdown by class (verified by running eval.py on all 100 images):

Real photos: 50 / 50 correct = 100.0%

Screen photos: 48 / 50 correct = 96.0%

The 2 incorrect predictions were screen images with extreme specular glare that made them visually ambiguous even to a human observer.

Dataset: 100 images (50 real, 50 screen). Real photos were taken of everyday objects. Screen photos were taken of a laptop screen displaying those same pictures - the exact cheating scenario described in the assignment.

3. Required Numbers: Latency & Cost per Image

Spot the Fake Photo - Technical Report

LATENCY

Inference latency (no GPU): ~21 ms on Laptop CPU (measured)

Measured on a standard Intel/AMD laptop CPU (no GPU needed):

- Pure inference (model already loaded in memory): ~21 ms average
- Full predict.py end-to-end including model load: ~99 ms (one-off startup)

MobileNetV3-Small has only 2.5M parameters and is 6 MB on disk.

It runs instantly and feels immediate to the end user.

COST PER IMAGE

On-device (mobile/edge): \$0.00 - completely free

Running on the user's own device is the best option:

- Zero cloud costs. Zero network latency. Works offline.
- Privacy-preserving: image never leaves the device.
- Model is small enough to ship inside a mobile app.

Cloud (AWS Lambda ARM64, 512 MB): ~\$0.89 per 1,000,000 images

Assumptions:

- Duration: 21 ms inference + ~30 ms overhead = ~51 ms per call
- AWS Lambda ARM64 price: \$0.0000000167 per ms
- Cost per image = 51 x \$0.0000000167 = \$0.00000085
- Per 1,000 images: ~\$0.001
- Per 1,000,000 images: ~\$0.89

On-device is preferred. The model is lightweight enough to run on any phone released after 2019 with no quality loss and no cost.

4. What I Would Improve with More Time

1. More Data: Collect 5,000+ images covering OLED/LCD/AMOLED screens, curved monitors, projectors, tablets, different refresh rates, and real objects in diverse lighting. More variety = better generalization.
2. Adversarial Robustness: As cheaters adapt (anti-Moire filters, high-quality prints), add adversarial examples to the training set. Regularly audit the model with new cheating techniques.
3. Edge Quantization: Apply INT8 post-training quantization (TorchScript/ONNX) to reduce model size to ~1.5 MB and latency to ~8 ms for mobile deployment.

Spot the Fake Photo - Technical Report

4. Cut-off Score Tuning: Use a held-out validation set to choose the optimal threshold (currently 0.5). A higher threshold (e.g. 0.7) reduces false positives - the right trade-off depends on business fraud risk.
5. Temporal Smoothing: In live camera mode, average over the last 3-5 frames to eliminate flickering and give more stable results.
6. k-Fold Cross-Validation: With only 100 images, report 5-fold CV accuracy for a statistically robust estimate.

Spot the Fake Photo - Technical Report

5. Live Web Demo Screenshot

The Streamlit web app (streamlit_app.py) provides the optional live demo:

- Upload tab: upload any image file for instant classification
- Camera tab: capture a live photo with the device camera

Screenshot below shows a real handbag photo correctly classified as 'REAL PHOTO' with 86.6% confidence in just 21 ms:

Spot the Fake Photo - Technical Report

Spot the Fake Photo

Upload an image — AI instantly tells you if it's a real photo or a screen screenshot.

 Upload Image  Take Photo

Choose an image file



Drag and drop file here

Limit 200MB per file • JPG, JPEG, PNG, WEBP

Browse files



20260706_154141.jpg

10.4KB



REAL PHOTO

Confidence: 86.6% | Latency: 21 ms

Powered by MobileNetV3-Small · Transfer Learning · Trained on 100 real images